

Secure Cloud Banking Infrastructure with Cybersecurity & Compliance Framework on AWS (Securebank-lite)

EXECUTIVE SUMMARY

Project: Secure Cloud Banking Infrastructure

Cloud Provider: AWS

Compliance: ISO 27001, NIST CSF

Project task and Deliverables:

Key Controls Implemented:

- IAM Least Privilege: Roles with minimum permissions, MFA enforced
- Network Security: VPC with public/private subnets, SGs, NACLs, WAF
- Encryption: AES-256 at rest, TLS 1.3 in transit
- Key Management: AWS KMS with automatic rotation (365 days)
- Monitoring: CloudTrail + GuardDuty + CloudWatch + Security Hub
- Incident Response: 4 playbooks (compromised key, EC2, data breach, DDoS)

Key Deliverables:

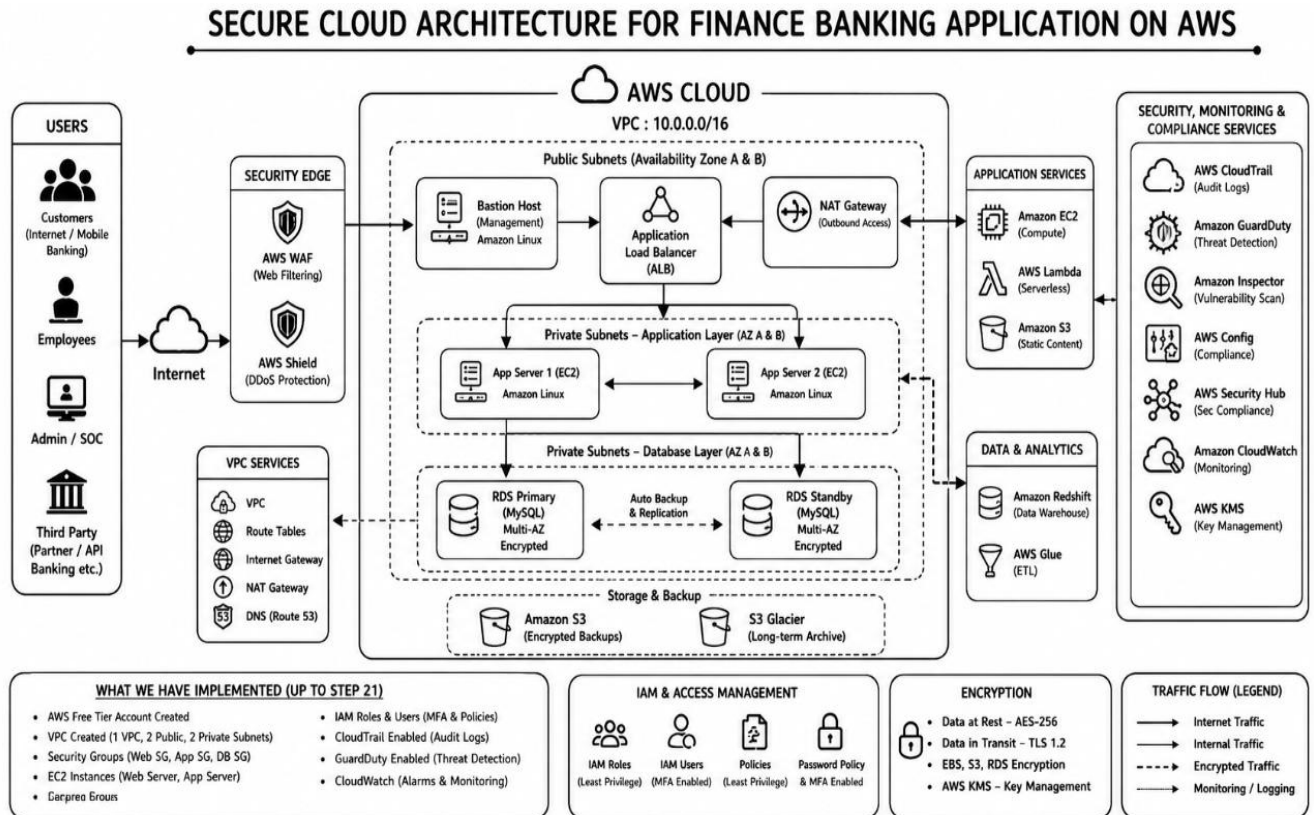
1. Security Architecture Document (detailed)
2. Incident Response Plan
3. Threat Model Diagrams (3)
4. Compliance Matrix (ISO + NIST)

Technology Stack Used

Area	Technology
Cloud	Amazon Web Services (AWS)
Monitoring	Amazon CloudWatch
Audit Logs	AWS CloudTrail
Threat Detection	Amazon GuardDuty
Key Management	AWS Key Management Service (KMS)
Identity & Access Management	AWS IAM
Diagram Tool	<u>Draw.io</u>
Documentation	Microsoft Word / Google Docs

System Architecture Diagram:

The Main Architecture



Project Introduction

This project shows the implementation of a secure cloud banking infrastructure using AWS services. In this project, security services like IAM, VPC, EC2, CloudWatch, CloudTrail, S3 encryption, and KMS are configured using AWS Free Tier. The aim of this project is to understand basic cloud security concepts with practical implementation.

1. Purpose

This document defines the security architecture for a cloud-hosted banking application. It describes how identity and access management, network security, encryption, key management, monitoring, and compliance are implemented using Amazon Web Services (AWS). The architecture follows the principles of least privilege, defence in depth, and zero trust.

2. Scope

The scope includes the following AWS services and components:

- Networking: Amazon VPC, public/private subnets, Security Groups, Network ACLs, Internet Gateway, NAT Gateway
- Compute: Amazon EC2 (web and application tier)
- Database: Amazon RDS (PostgreSQL)
- Storage: Amazon S3 (backups and logs)
- Load balancing: Application Load Balancer (ALB)
- Web firewall: AWS WAF
- Identity: AWS IAM (roles, policies, Identity Center)
- Key management: AWS KMS
- Logging & monitoring: AWS CloudTrail, CloudWatch, GuardDuty, Security Hub
- External access: End users connecting over the internet

Project 1 - Secure S3 Bucket with Encryption

In this project, an S3 bucket was created and encryption was enabled using AWS KMS. A bucket policy was also applied to deny uploads without encryption. Testing was performed to verify access control.

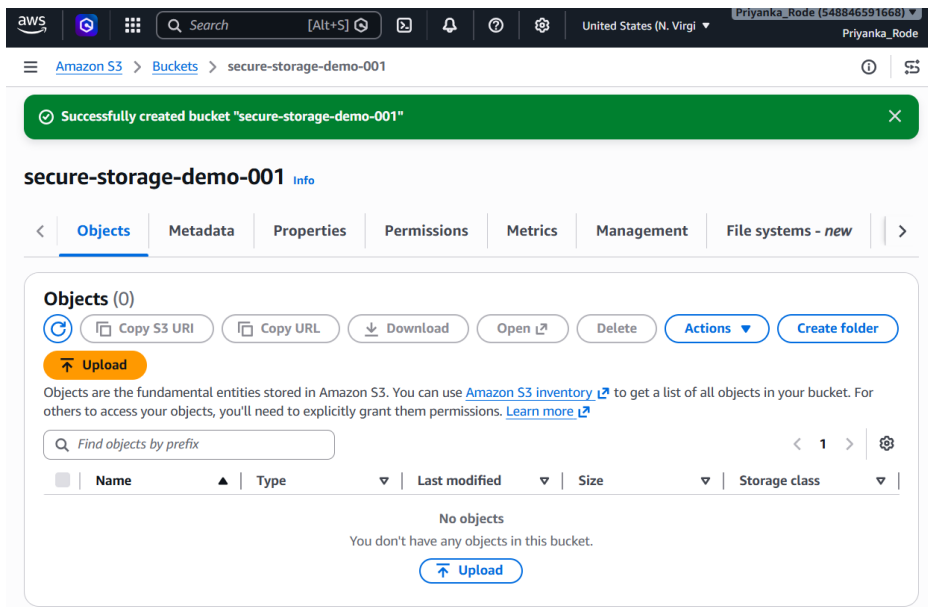


Figure 1: Creating bucket in S3

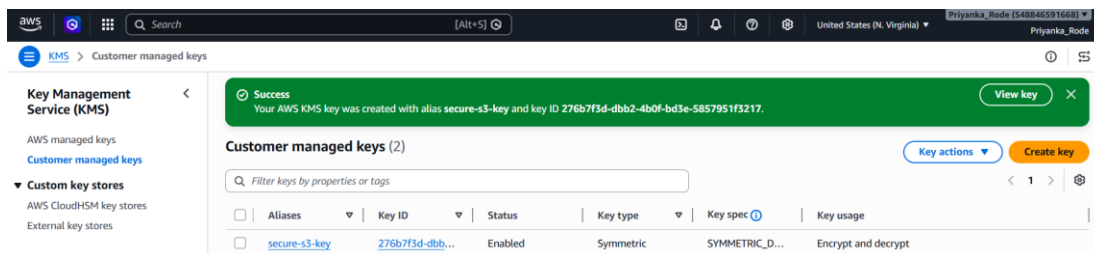


Figure 2: Creating key in KMS

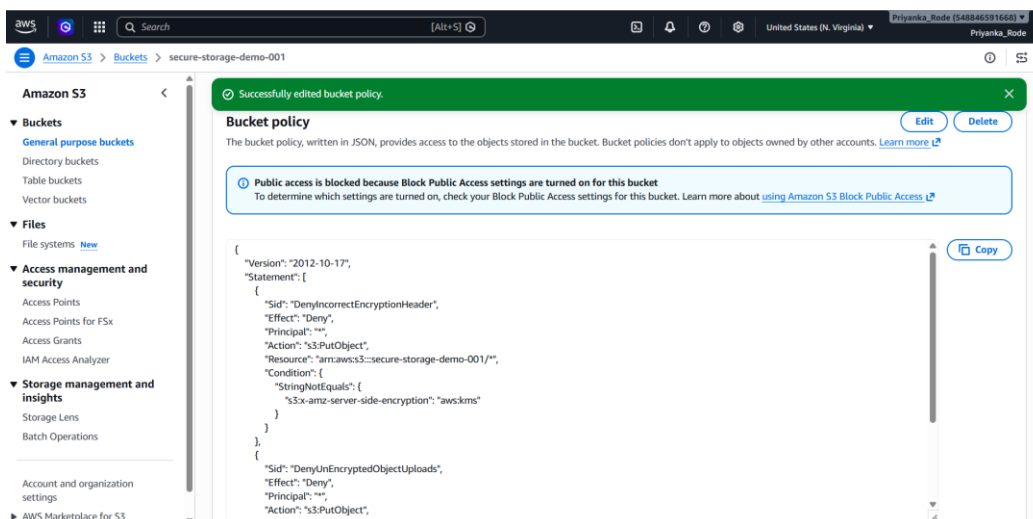


Figure 3: Editing Bucket Policy

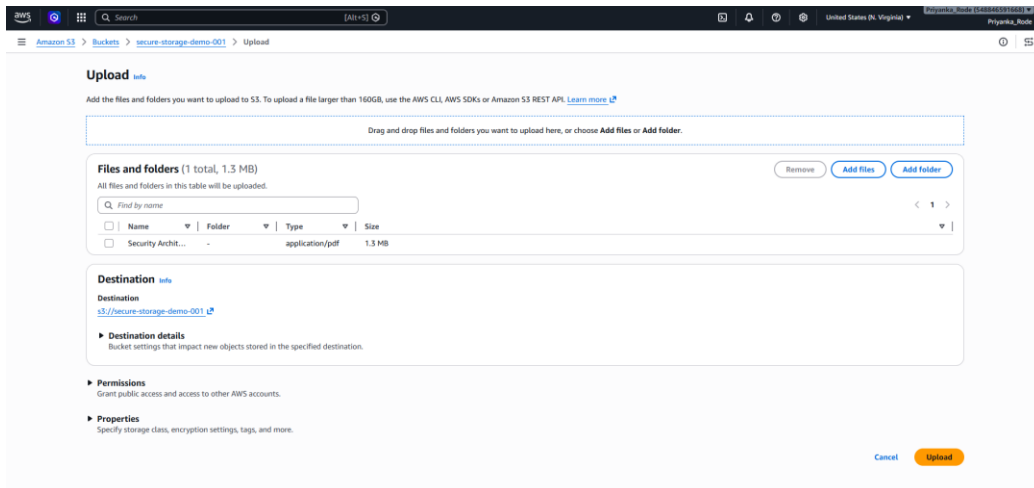


Figure 4: Uploading file in S3

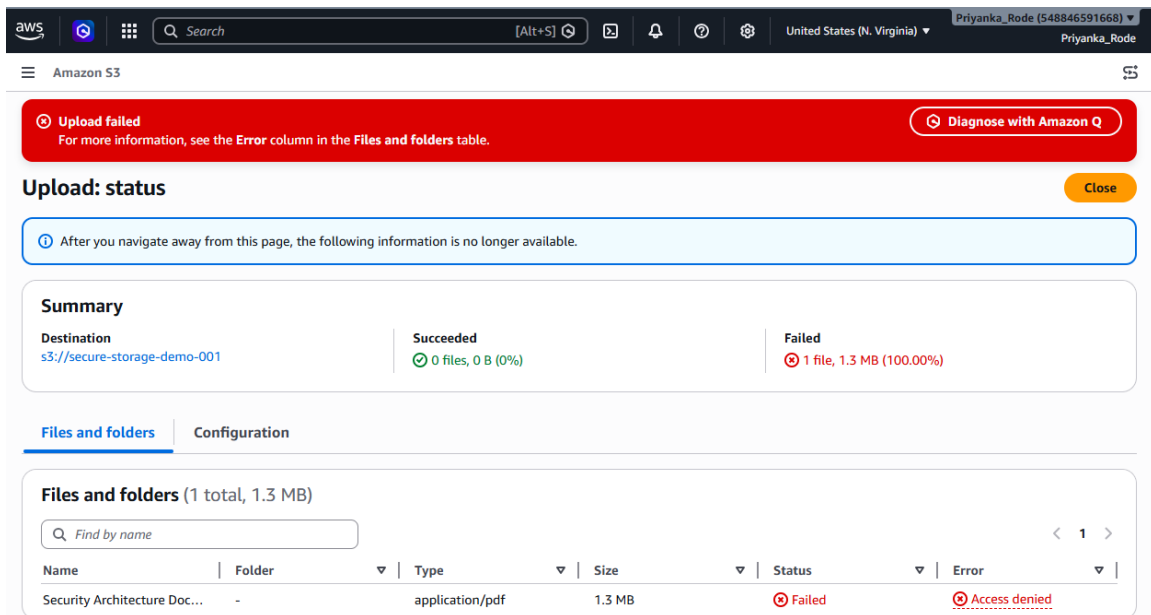


Figure 4: files should fail due to encryption

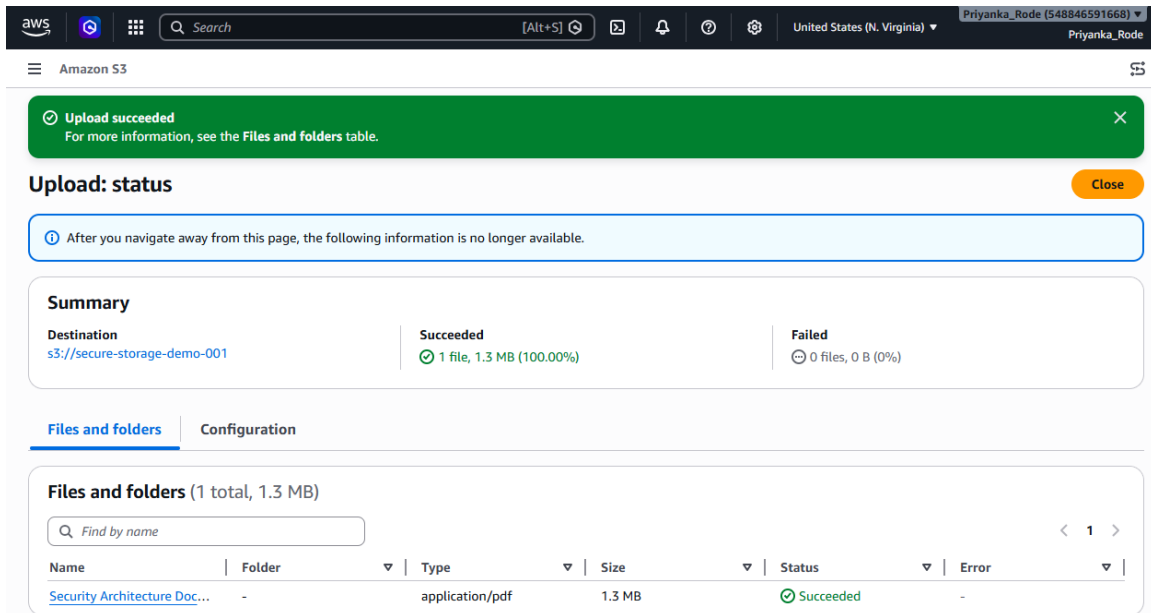
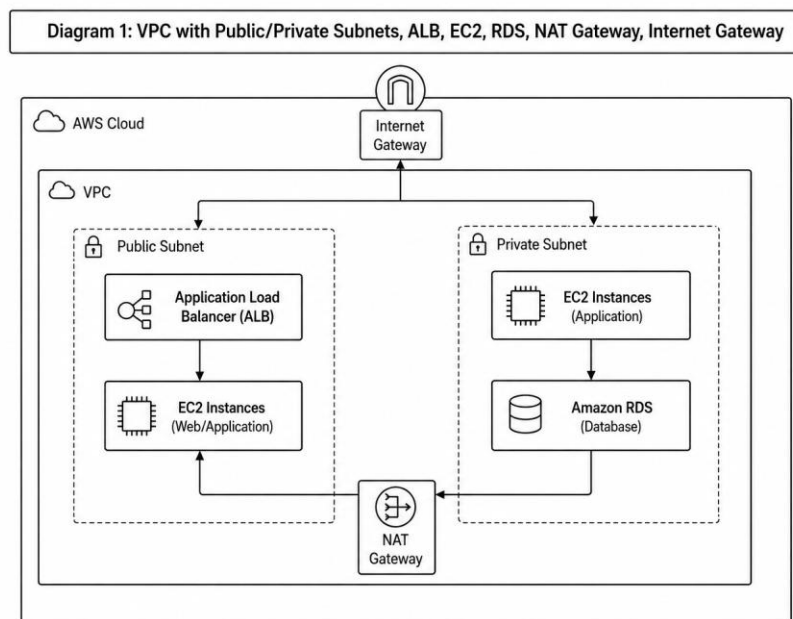


Figure 5: Editing rules and successfully uploading file

Project 2 - Create Secure VPC and EC2 Infrastructure

In this project, a custom VPC was created with public and private subnets. Security groups were configured for restricted SSH access. A secure EC2 instance was launched inside the private subnet.

Architecture:



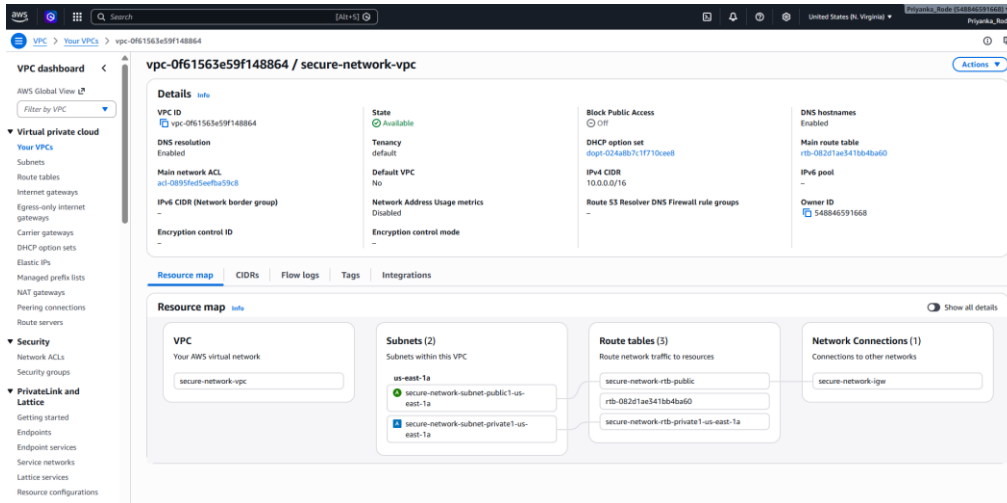


Figure 6: Creating VPC

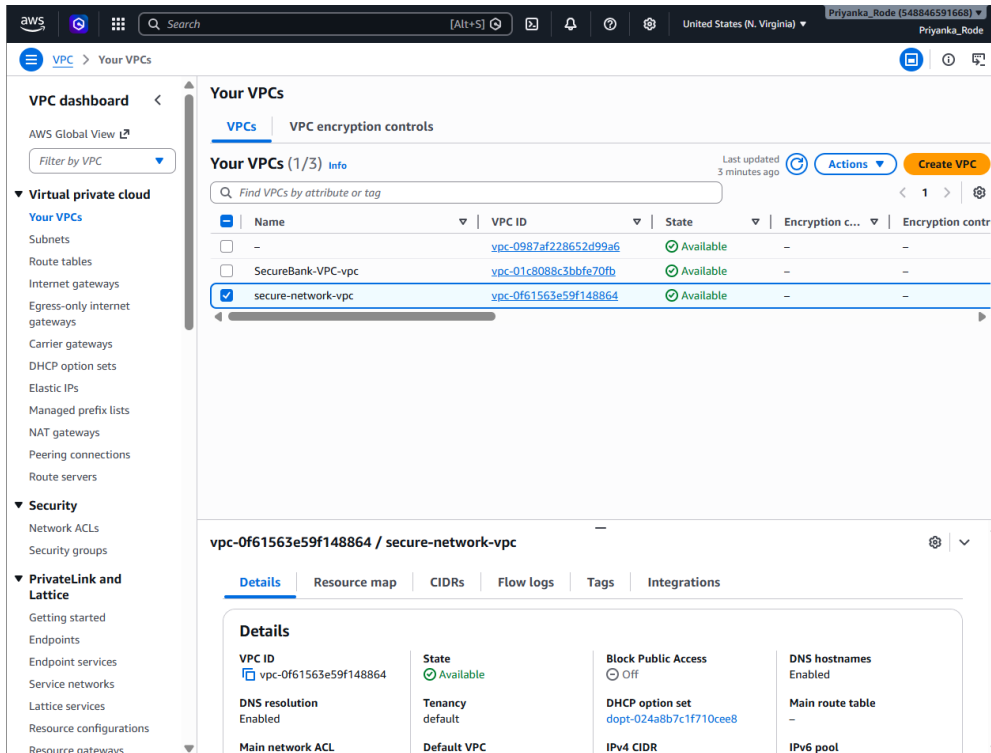


Figure 7: Our VPC

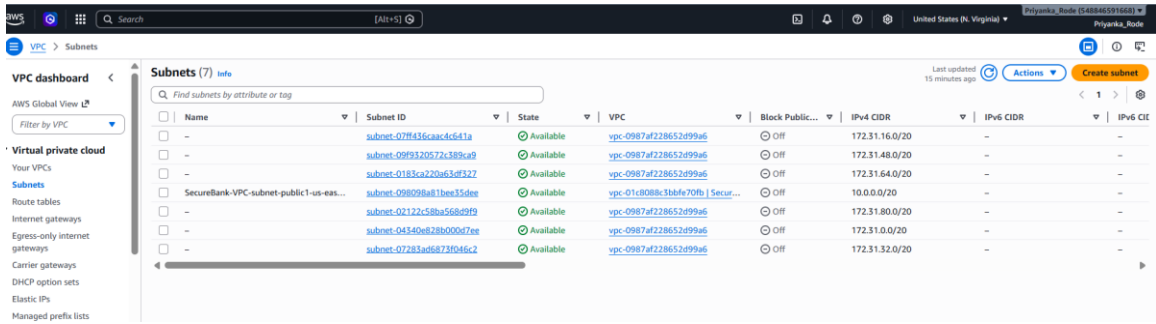


Figure 8: Creating Subnet: private and public

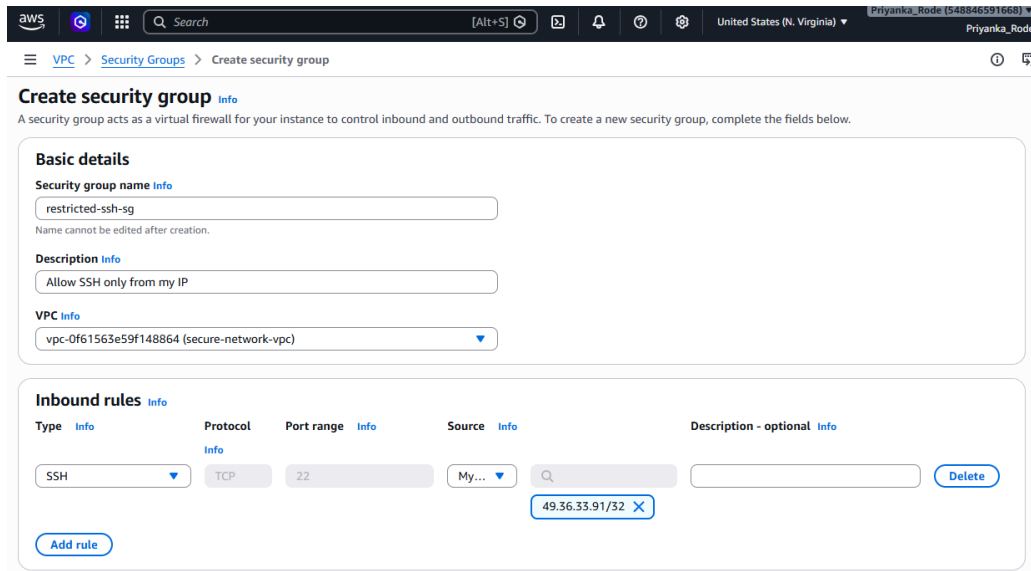


Figure 9: Creating Security Group: SSH/HTTP

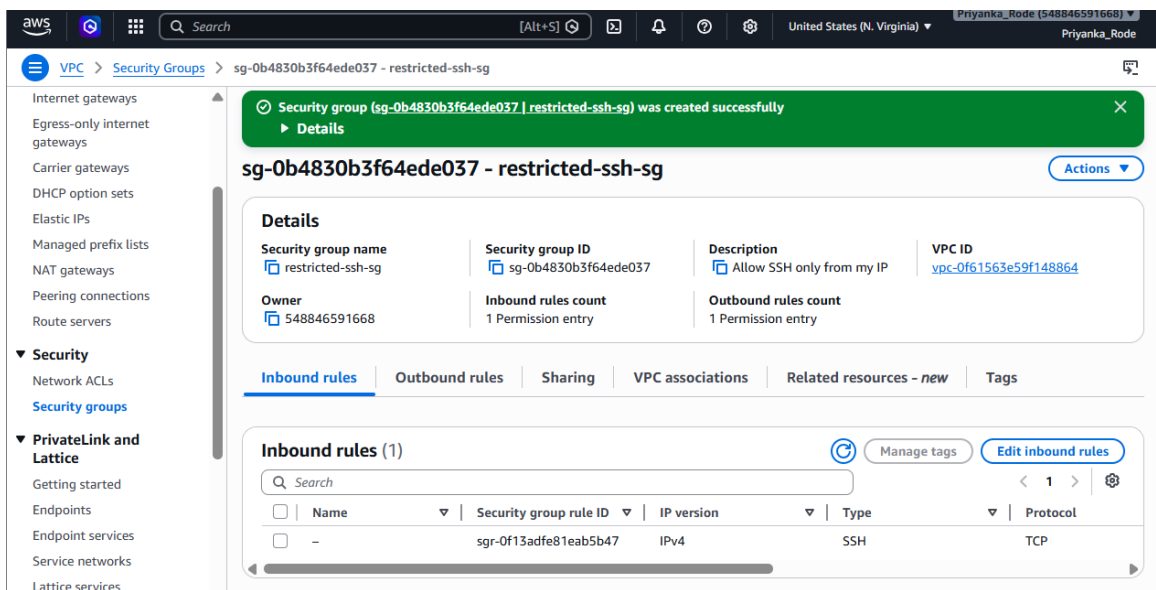


Figure 10: SG created

Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

secure-keypair [Create new key pair](#)

Network settings Info

VPC - *required* Info

vpc-0f61563e59f148864 (secure-network-vpc) 10.0.0.0/16 [Create new VPC](#)

Subnet Info

subnet-0dd8f214e2c921575 [Create new subnet](#)

secure-network-subnet-private1-us-east-1a

VPC: vpc-0f61563e59f148864 Owner: 548846591668

Availability Zone: us-east-1a (use1-az1) Zone type: Availability Zone

IP addresses available: 4091 CIDR: 10.0.128.0/20

Auto-assign public IP Info

Disable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☐ Create security group ☒ Select existing security group

Common security groups Info

Select security groups [Compare security group rules](#)

restricted-ssh-sg sg-0b4830b3f64ede037 [X](#)

VPC: vpc-0f61563e59f148864

Security groups that you add or remove here will be added to or removed from all your network interfaces.

Summary

Number of instances [Info](#)

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.11... [read more](#)

ami-00e801948462f718a

Virtual server type (instance type)

t3.micro

Firewall (security group)

restricted-ssh-sg

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch Instance](#) [Preview code](#)

Figure

11: Launching EC2 from private Subnet

Success

Successfully initiated launch of instance (i-0d893defa840a886)

Launch log

Next Steps

Q. What would you like to do next with this instance, for example "create alarm" or "create backup"

Create billing usage alerts

To manage costs and avoid surprise bills, set up email notifications for billing usage thresholds.

[Create billing alerts](#)

Connect to your instance

Once your instance is running, log into it from your local computer.

[Connect to instance](#)

[Learn more](#)

Connect an RDS database

Configure the connection between an EC2 instance and a database to allow traffic flow between them.

[Connect an RDS database](#)

[Create a new RDS database](#)

[Learn more](#)

Create EBS snapshot policy

Create a policy that automates the creation, retention, and deletion of EBS snapshots

[Create EBS snapshot policy](#)

Manage detailed monitoring

Enable or disable detailed monitoring for the instance. If you enable detailed monitoring, the Amazon EC2 console displays monitoring graphs with a 1-minute period.

[Manage detailed monitoring](#)

Create Load Balancer

Create an application, network gateway or classic Elastic Load Balancer

[Create Load Balancer](#)

Create AWS budget

AWS Budgets allows you to create budgets, forecast spend, and take action on your costs and usage from a single location.

[Create AWS budget](#)

Manage CloudWatch alarms

Create or update Amazon CloudWatch alarms for the instance.

[Manage CloudWatch alarms](#)

Figure 12: Launched Successfully

Project 3 - IAM Least Privilege Access

In this project, an IAM user and custom policy were created. The user was given limited S3 access permissions. AWS CLI was used to test allowed and denied operations.

Example IAM Policy – Web Server Role

```
{
  "Id": "key-consolepolicy-3",
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Enable IAM User Permissions",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::548846591668:root"
      },
      "Action": "kms:*",
      "Resource": "*"
    }
  ]
}
```

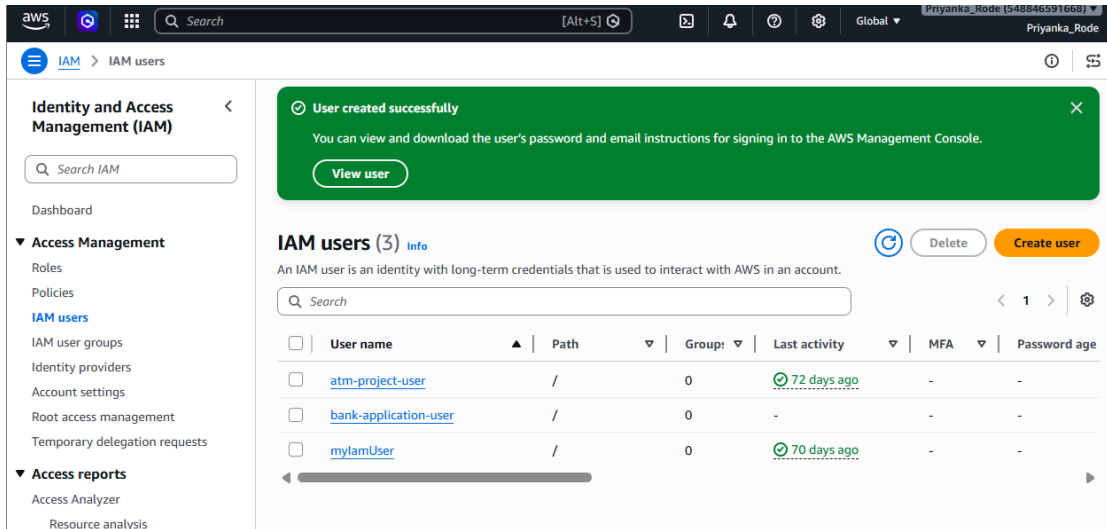


Figure 13: IAM Created

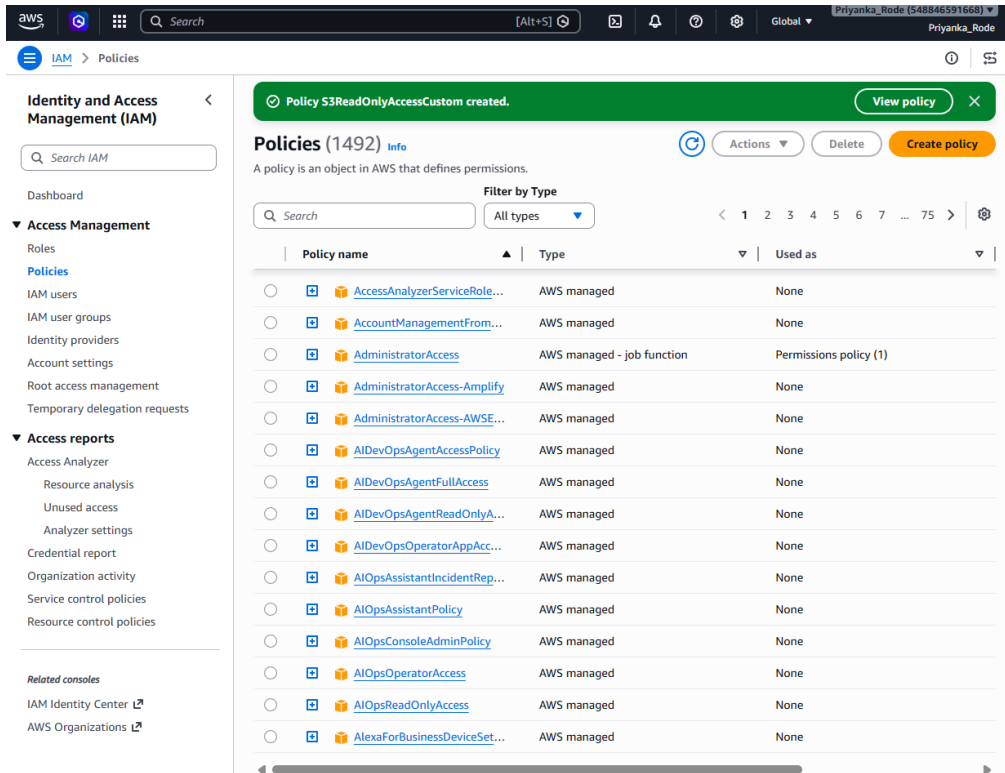


Figure 13: Updating Policies

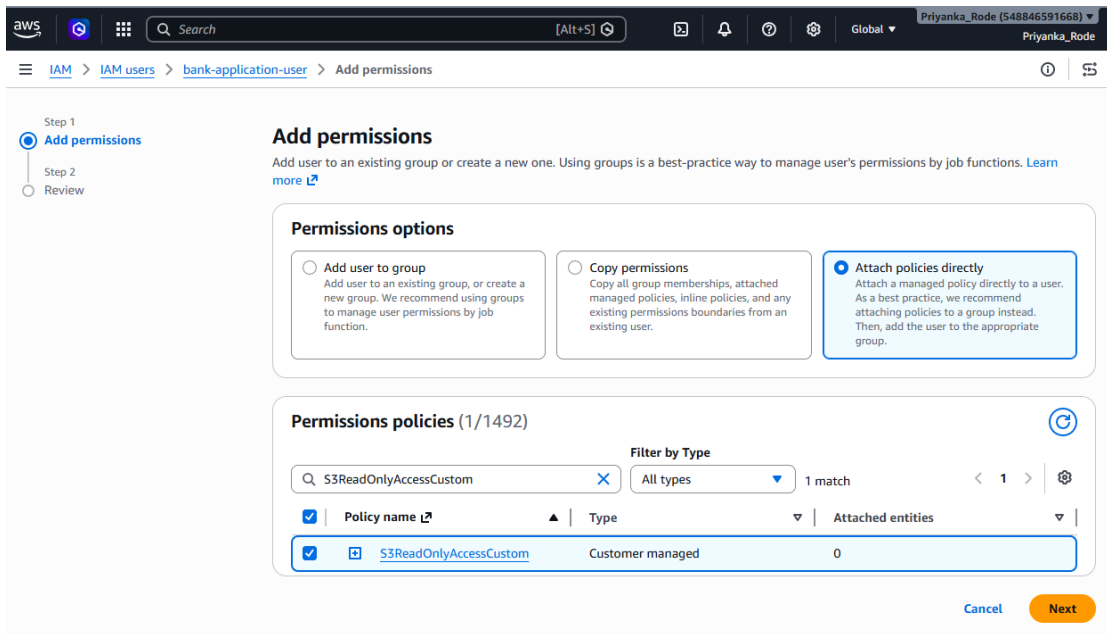


Figure 14: Adding S3ReadOnlyAccessCustom rule

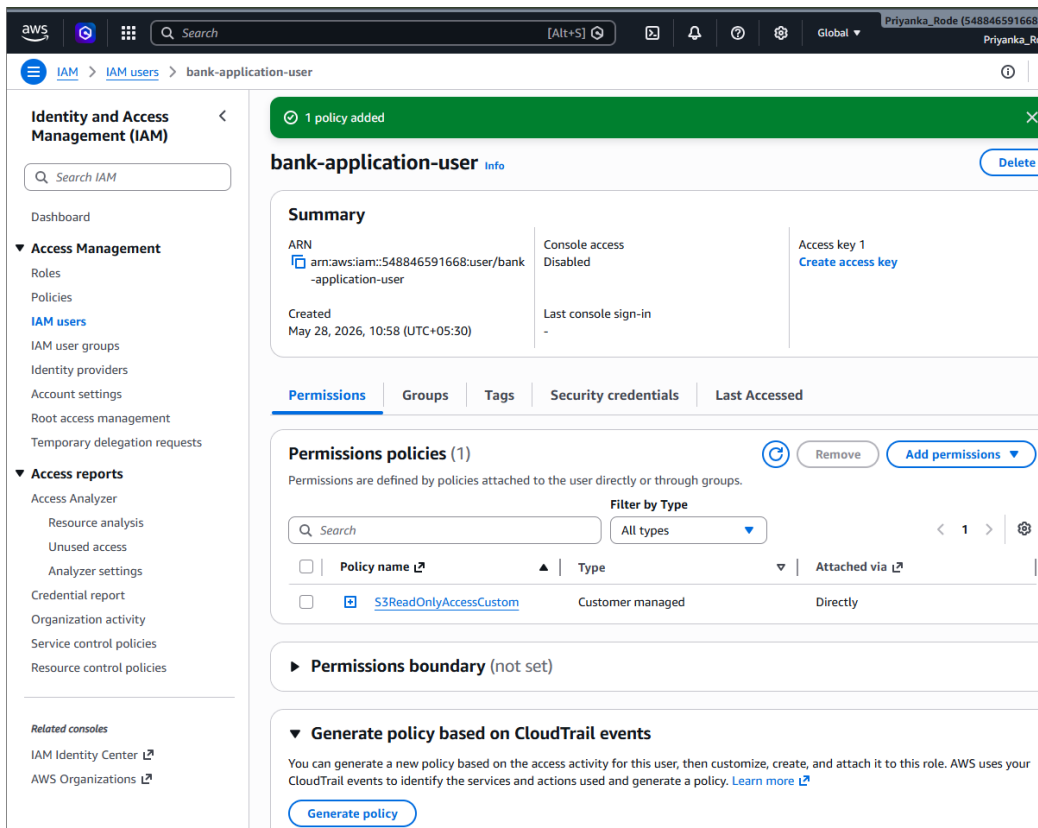


Figure 15: policy added

- Some cmd commands

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rodep>aws configure
AWS Access Key ID [*****L2MH]: AKIAX7S00L22PFW7H2MY
AWS Secret Access Key [*****E3Gz]: 5sqZqsV2bIBo10eRBnjLs6WPXtPeI1FtgChjvEZ8
Default region name [ap-south-1]: us-east-1
Default output format [json]: json

C:\Users\rodep>
```

```
C:\Users\rodep>aws s3 ls

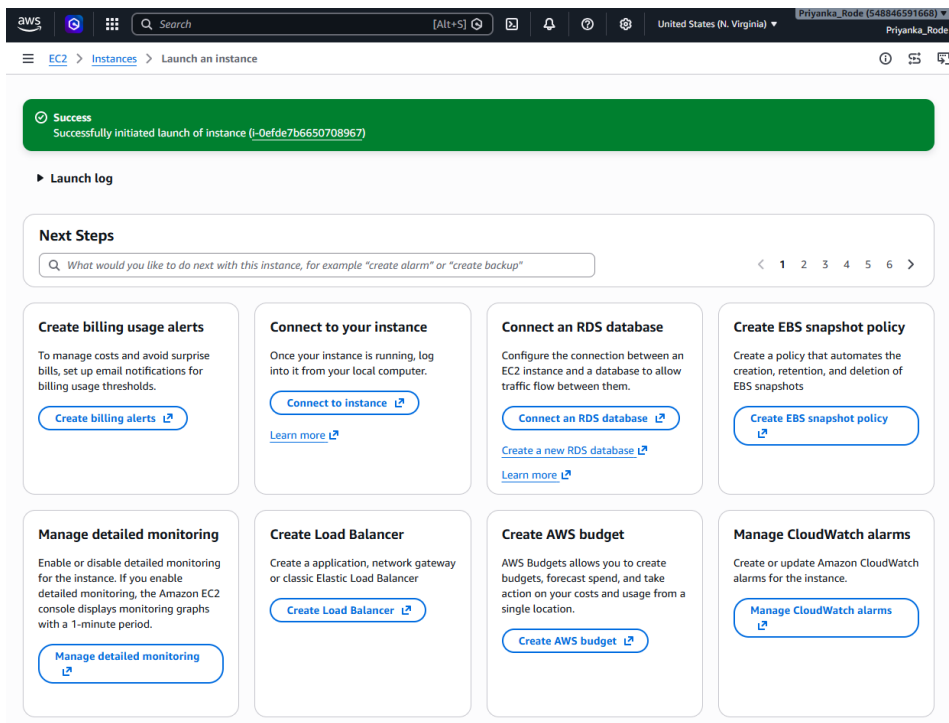
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListBuckets operation: User: arn:aws:iam::548846591668:user/bank-application-user is not authorized to perform: s3:ListAllMyBuckets because no identity-based policy allows the s3:ListAllMyBuckets action

C:\Users\rodep>|
```

```
C:\Users\rodep>aws s3 cp s3://secure-storage-demo-001/test.txt .
download: s3://secure-storage-demo-001/test.txt to .\test.txt
```

Project 4 - Launch EC2 Web Server

An Amazon Linux EC2 instance was launched and connected using SSH. Apache web server was installed and a simple secure banking webpage was hosted.



- **Commands for Launching EC2 on Browser:**

```
[ec2-user@ip-10-0-4-219 ~]$ sudo yum install httpd -y
Last metadata expiration check: 0:00:17 ago on Thu May 28 07:59:14 2026.
Dependencies resolved.
=====
Package                Architecture Version                Repository            Size
=====
Installing:
httpd                  x86_64          2.4.67-1.amzn2023.0.1  amazonlinux           46 k
Installing dependencies:
apr                    x86_64          1.7.5-1.amzn2023.0.4   amazonlinux           129 k
apr-util              x86_64          1.6.3-1.amzn2023.0.2   amazonlinux           97 k
apr-util-ldap         x86_64          1.6.3-1.amzn2023.0.2   amazonlinux           13 k
generic-logos-httpd   noarch          18.0.0-12.amzn2023.0.3 amazonlinux            19 k
httpd-core             x86_64          2.4.67-1.amzn2023.0.1  amazonlinux           1.4 M
httpd-filesystem       noarch          2.4.67-1.amzn2023.0.1  amazonlinux            12 k
httpd-tools            x86_64          2.4.67-1.amzn2023.0.1  amazonlinux            80 k
libbrotli              x86_64          1.0.9-4.amzn2023.0.2   amazonlinux           315 k
mailcap                noarch          2.1.49-3.amzn2023.0.3  amazonlinux            33 k
Installing weak dependencies:
apr-util-openssl       x86_64          1.6.3-1.amzn2023.0.2   amazonlinux            15 k
mod_http2              x86_64          2.0.39-1.amzn2023.0.1  amazonlinux           167 k
mod_lua                x86_64          2.4.67-1.amzn2023.0.1  amazonlinux            59 k

Transaction Summary
=====
Install 13 Packages

Total download size: 2.4 M
Installed size: 7.0 M
Downloading Packages:
(1/13): apr-util-1.6.3-1.amzn2023.0.2.x86_64.rpm                2.4 MB/s | 97 kB    00:00
(2/13): apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64.rpm           313 kB/s | 13 kB    00:00
(3/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm   751 kB/s | 19 kB    00:00
(4/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm                     1.7 MB/s | 129 kB   00:00
(5/13): httpd-2.4.67-1.amzn2023.0.1.x86_64.rpm                 1.6 MB/s | 46 kB    00:00
(6/13): apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64.rpm        255 kB/s | 15 kB    00:00
(7/13): httpd-core-2.4.67-1.amzn2023.0.1.x86_64.rpm             34 MB/s | 1.4 MB    00:00
(8/13): httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch.rpm        422 kB/s | 12 kB    00:00
(9/13): httpd-tools-2.4.67-1.amzn2023.0.1.x86_64.rpm            2.5 MB/s | 80 kB    00:00
(10/13): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm              9.1 MB/s | 315 kB   00:00
(11/13): mailcap-2.1.49-3.amzn2023.0.3.noarch.rpm                1.2 MB/s | 33 kB    00:00
(12/13): mod_http2-2.0.39-1.amzn2023.0.1.x86_64.rpm             5.2 MB/s | 167 kB   00:00
(13/13): mod_lua-2.4.67-1.amzn2023.0.1.x86_64.rpm               2.2 MB/s | 59 kB    00:00
-----
Total                                                                10 MB/s | 2.4 MB    00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing : 1/1
Installing : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Installing : apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64 2/13
Installing : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 3/13
Installing : apr-util-1.6.3-1.amzn2023.0.2.x86_64 4/13
Installing : mailcap-2.1.49-3.amzn2023.0.3.noarch 5/13
Installing : httpd-tools-2.4.67-1.amzn2023.0.1.x86_64 6/13
Installing : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 7/13
Running scriptlet: httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch 8/13
Installing : httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch 8/13
```

```
[ec2-user@ip-10-0-4-219 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-4-219 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-4-219 ~]$ echo "<h1>Finance Bank Secure Web Server</h1>" | sudo tee /var/www/html/index.html
<h1>Finance Bank Secure Web Server</h1>
[ec2-user@ip-10-0-4-219 ~]$ echo "<h1>SecureBank-lite Web Server</h1>" | sudo tee /var/www/html/index.html
<h1>SecureBank-lite Web Server</h1>
[ec2-user@ip-10-0-4-219 ~]$ |
```


SecureBank-lite Web Server

Figure 17: Final output on Browser

Project 5 - CloudWatch Monitoring Dashboard

CloudWatch dashboard and monitoring metrics were created to monitor EC2 instance performance and network usage.

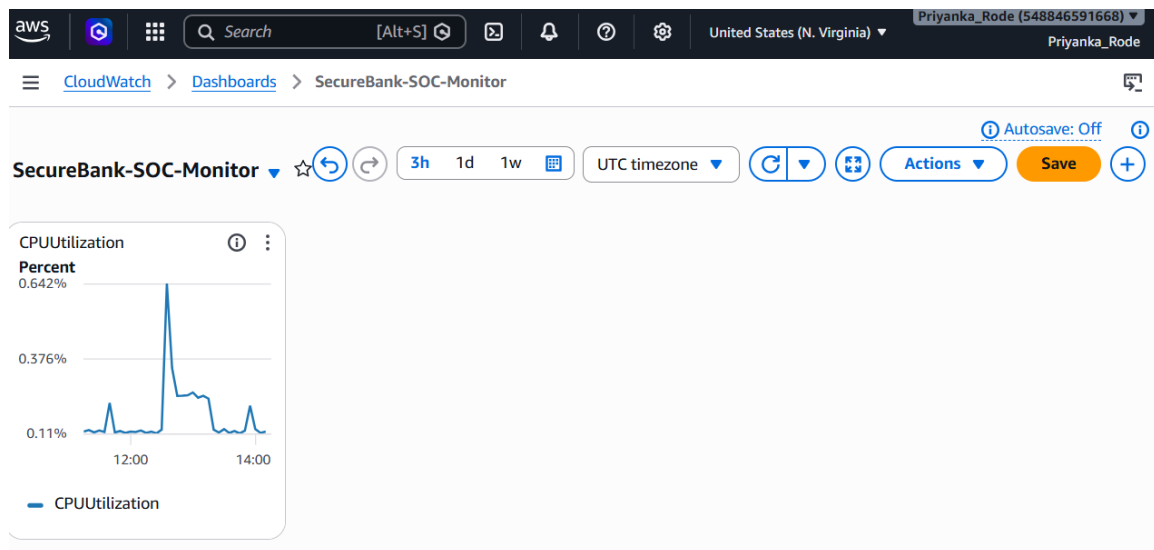


Figure 18: CloudWatch : CPUUtilization

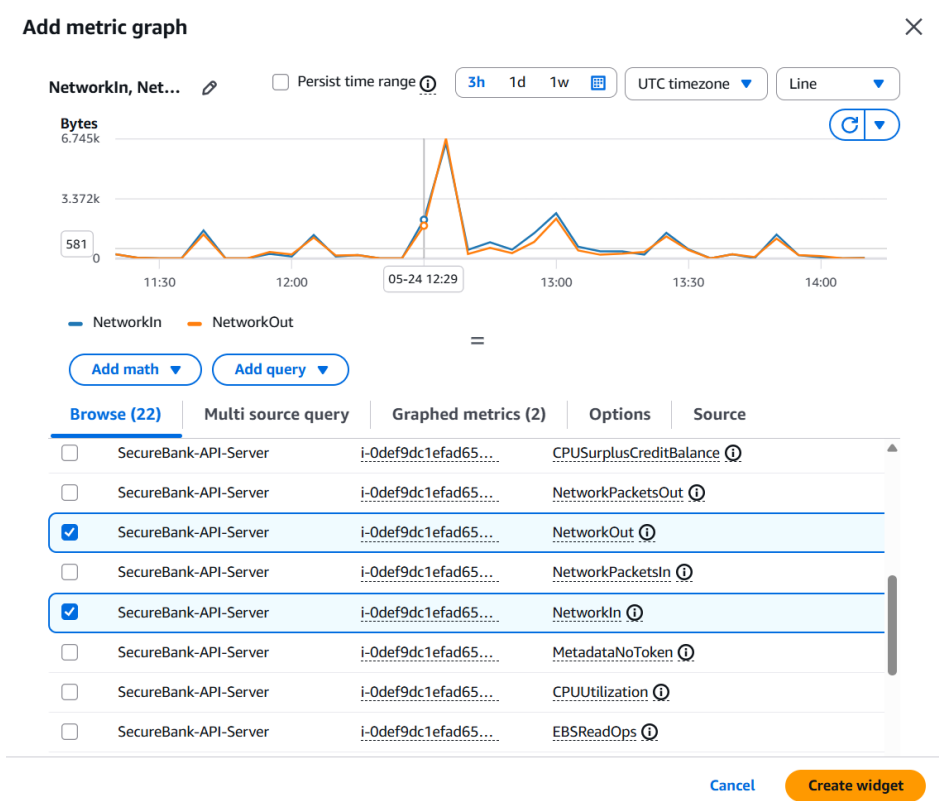


Figure 19: Network IN/OUT

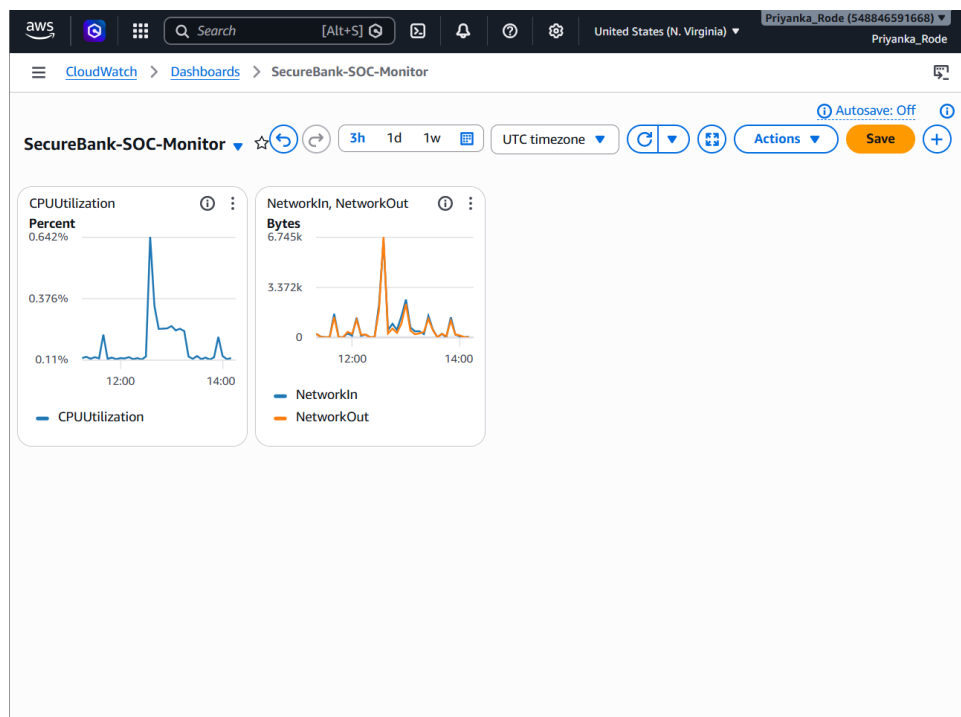
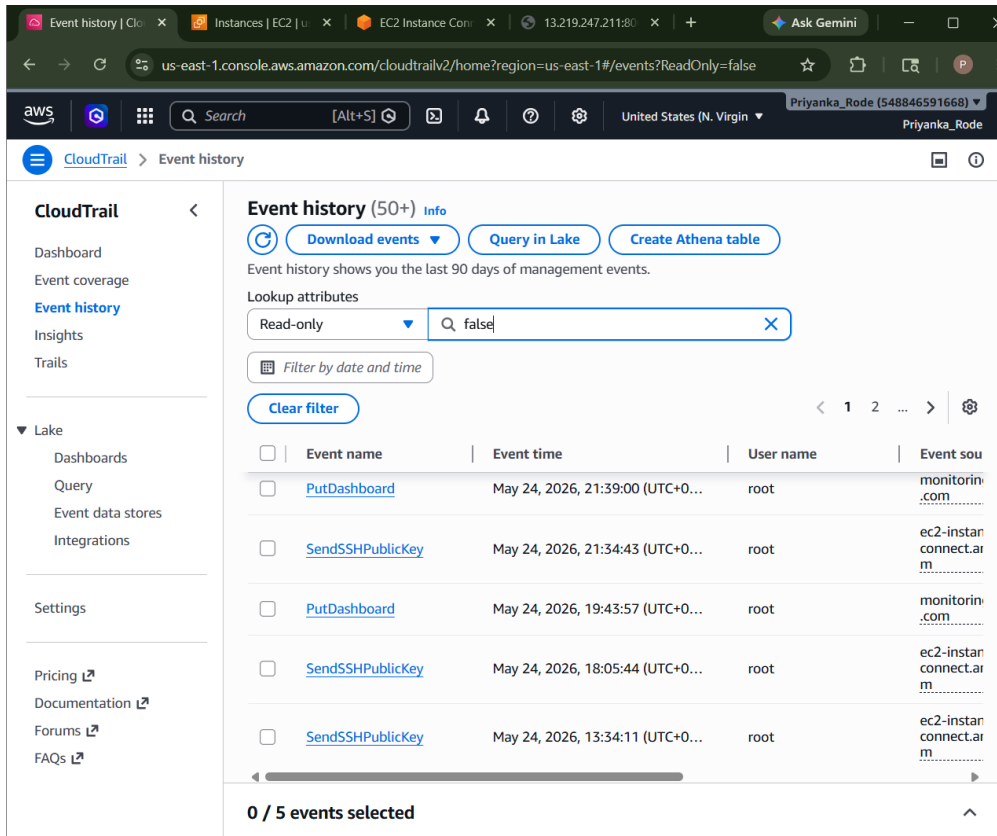


Figure 20: SOC Dashboard

Project 6 - CloudTrail Monitoring

CloudTrail was enabled to monitor AWS account activities such as API calls and login events.



The screenshot displays the AWS CloudTrail console's 'Event history' page. The left sidebar contains navigation links for CloudTrail, Dashboard, Event coverage, Event history (selected), Insights, Trails, Lake (Dashboards, Query, Event data stores, Integrations), Settings, Pricing, Documentation, Forums, and FAQs. The main content area shows 'Event history (50+)' with buttons for 'Download events', 'Query in Lake', and 'Create Athena table'. It includes a 'Lookup attributes' section with a dropdown set to 'Read-only' and a search box containing 'false'. Below this is a 'Filter by date and time' button and a 'Clear filter' button. A table lists five events, each with a checkbox, event name, time, user name, and source. The events are: 'PutDashboard' (monitoring.com), 'SendSSHPublicKey' (ec2-instanceconnect.amazonaws.com), 'PutDashboard' (monitoring.com), 'SendSSHPublicKey' (ec2-instanceconnect.amazonaws.com), and 'SendSSHPublicKey' (ec2-instanceconnect.amazonaws.com). At the bottom, it indicates '0 / 5 events selected'.

☐	Event name	Event time	User name	Event source
☐	PutDashboard	May 24, 2026, 21:39:00 (UTC+0...)	root	monitoring.com
☐	SendSSHPublicKey	May 24, 2026, 21:34:43 (UTC+0...)	root	ec2-instanceconnect.amazonaws.com
☐	PutDashboard	May 24, 2026, 19:43:57 (UTC+0...)	root	monitoring.com
☐	SendSSHPublicKey	May 24, 2026, 18:05:44 (UTC+0...)	root	ec2-instanceconnect.amazonaws.com
☐	SendSSHPublicKey	May 24, 2026, 13:34:11 (UTC+0...)	root	ec2-instanceconnect.amazonaws.com

Figure 21: CloudTrail

Conclusion

This project successfully demonstrated the implementation of a secure cloud banking environment using AWS services. Security controls like encryption, IAM least privilege, network security, EC2 hosting, monitoring, and logging were configured practically. The project helped in understanding real-world cloud security implementation using simple AWS services.

Document Control

Field	Information
Document Title	Security Architecture Document – Secure Cloud Banking Application
Version	1.0
Date	28-05-2026
Author	Priyanka Anil Rode